

TEMAS SELECTOS DE CÓMPUTO DE ALTO RENDIMIENTO PARA SIMULACIONES FÍSICAS

Reporte de Simulación de Ecuación de Calor 2D.

Saúl Ruano Sánchez

Semestre 2026-2

Resumen

Se estudia el uso de herramientas de paralelización para implementar un código que resuelve la ecuación de calor en su caso estacionario. Se realizan pruebas de rendimiento entre distintas APIs de paralelización, utilizando parámetros como tiempos de ejecución y eficiencia con múltiples hilos de trabajo en una CPU.

1. Introducción

El cómputo de alto rendimiento ha experimentado una transformación significativa con el surgimiento de APIs de paralelización como OpenMP [1] y OpenACC [2], que permiten aprovechar de forma eficiente los recursos de hardware actuales, procesadores multinúcleo y unidades de procesamiento gráfico (GPU), sin necesidad de reescribir por completo el código base.

En este trabajo se evalúa el rendimiento de ambas APIs al resolver la ecuación de Laplace en dos dimensiones mediante el método de diferencias finitas, comparando tiempos de ejecución, speedup y eficiencia en función del número de hilos. El objetivo es determinar cuál de las dos herramientas se adapta mejor a arquitecturas de CPU para este tipo de problemas, y discutir las implicaciones de cada enfoque en un contexto de simulación física.

2. Fundamentos y teoría

2.1. Ecuación de Calor

El flujo de calor por difusión en un medio continuo se rige por la ley de Fourier, que establece que el flujo de calor $\vec{J}_q$ es proporcional al gradiente negativo de la temperatura,

$$\vec{J}_q = -\kappa \nabla T, \quad (1)$$

donde κ representa la conductividad térmica del material. Al combinar (1), con la ecuación de continuidad de la energía interna, en ausencias de fuentes, obtenemos la ecuación de calor

$$\begin{aligned} \rho \partial_t U &= \rho c_p \partial_t T \\ &= -\nabla \cdot \vec{J}_q = k \nabla^2 T, \end{aligned} \quad (2)$$

de donde $\partial_t T = \alpha \nabla^2 T$ con $\alpha = k/(\rho c_p)$. Nos centramos en el régimen estacionario para una y dos dimensiones sobre un dominio rectangular Ω , con distintas condiciones en la frontera $\partial\Omega$. En este caso, la ecuación (2) toma la forma de una ecuación de Laplace,

$$\nabla^2 T = 0. \quad (3)$$

2.2. Condiciones de Frontera

Para la ecuación de Laplace, consideramos dos tipos de condiciones sobre la frontera $\partial\Omega$ del dominio rectangular:

Condiciones tipo Dirichlet: se impone el valor de la temperatura sobre la frontera,

$$T(\mathbf{x}) = T_D(\mathbf{x}), \quad \mathbf{x} \in \partial\Omega_D, \quad (4)$$

donde T_D es una función conocida, para nuestro caso la tomaremos como constante, y $\partial\Omega_D$ denota la porción de la frontera donde se aplica la condición.

Condiciones tipo Neumann: se impone el valor del flujo normal de calor sobre la frontera,

$$-k \frac{\partial T}{\partial n} = q_N(\mathbf{x}), \quad \mathbf{x} \in \partial\Omega_N, \quad (5)$$

o bien, de forma equivalente, la derivada normal de la temperatura:

$$\frac{\partial T}{\partial n} = g_N(\mathbf{x}), \quad \mathbf{x} \in \partial\Omega_N, \quad (6)$$

donde $\partial/\partial n$ denota la derivada en la dirección normal exterior a la frontera, q_N es el flujo dado.

2.3. Diferencias Finitas

Para resolver numéricamente la ecuación es necesario discretizar su dominio. El método de diferencias finitas logra esto aproximando las derivadas mediante desarrollos en serie de Taylor de la función que se quiere resolver [3, 4].

2.3.1. Caso 1D

Partiendo de la ecuación (3), que en una dimensión tiene la siguiente forma,

$$\frac{d^2 T}{dx^2} = 0, \quad (7)$$

para aproximar la segunda derivada, recurrimos al desarrollo en serie de Taylor de la función T en torno a x , donde Δx es el espaciamiento de la malla sobre la que se resuelve la ecuación. En la Figura 1 se muestra la discretización resultante.

$$\frac{d^2 T}{dx^2} \approx \frac{T_{i+1} - 2T_i + T_{i-1}}{(\Delta x)^2} = 0. \quad (8)$$

donde adoptamos la notación, $T_i \equiv T(x_i)$, $T_{i\pm 1} \equiv T(x_i \pm \Delta x)$. Se observa que el problema continuo se reduce a uno lineal

$$T_{i+1} - 2T_i + T_{i-1} = 0, \quad i = 1, \dots, N-2, \quad (9)$$

donde N es el número total de nodos en la malla. Las ecuaciones en los nodos de frontera ($i = 0$) y ($i = N-1$) se sustituyen por las condiciones de frontera correspondientes.

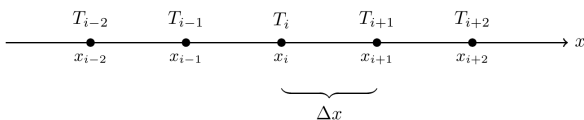


Figura 1: Discretización del espacio.

Suponiendo que queremos resolver la ecuación de Laplace en el intervalo $[0, L]$, con condiciones de frontera de Dirichlet, de tal forma que fijamos el valor de la temperatura en los extremos,

$$T(0) = T_0, \quad T(L) = T_L, \quad (10)$$

para ejemplificar el método, discretizamos el dominio en $N = 6$ nodos, con $\Delta x = L/(N-1)$. La ecuación discretizada (9) se aplica en los nodos interiores $i = 1, 2, 3, 4$, mientras que en los nodos de frontera imponemos directamente los valores dados por las condiciones de frontera. De tal forma que obtenemos el siguiente sistema de ecuaciones,

$$\begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & -2 & 1 & 0 & 0 & 0 \\ 0 & 1 & -2 & 1 & 0 & 0 \\ 0 & 0 & 1 & -2 & 1 & 0 \\ 0 & 0 & 0 & 1 & -2 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} T_0 \\ T_1 \\ T_2 \\ T_3 \\ T_4 \\ T_5 \end{pmatrix} = \begin{pmatrix} T_0 \\ 0 \\ 0 \\ 0 \\ 0 \\ T_L \end{pmatrix}. \quad (11)$$

Si en cambio se imponen condiciones de frontera de Neumann, se fija el valor del flujo en los extremos del dominio. En este caso, las ecuaciones de los nodos de frontera ya no son triviales: la derivada en $i = 0$ se aproxima mediante una diferencia adelantada, y la derivada en $i = N-1$ mediante una diferencia atrasada, lo que modifica las primeras y últimas filas del sistema manteniendo inalteradas las ecuaciones de los nodos interiores.

2.3.2. Caso 2D

Partiendo de igual forma de la ecuación (3),

$$\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} = 0. \quad (12)$$

Al igual que en el caso 1D, discretizamos el dominio de la ecuación empleando una malla rectangular uniforme con espaciamientos Δx y Δy , y adoptamos la notación $T_{i,j} \equiv T(x_i, y_j)$ con $x_i = i \Delta x$, $y_j = j \Delta y$. Aplicando la diferencia centrada de segundo orden a cada una de las derivadas parciales se obtiene la aproximación de cinco puntos

$$\frac{T_{i+1,j} - 2T_{i,j} + T_{i-1,j}}{\Delta x^2} + \frac{T_{i,j+1} - 2T_{i,j} + T_{i,j-1}}{\Delta y^2} = 0. \quad (13)$$

En el caso particular de mallas con $\Delta x = \Delta y \equiv h$, la expresión anterior se simplifica al *stencil de*

cinco puntos

$$T_{i+1,j} + T_{i-1,j} + T_{i,j+1} + T_{i,j-1} - 4T_{i,j} = 0, \\ i = 1, \dots, N_x - 2, j = 1, \dots, N_y - 2, \quad (14)$$

donde N_x y N_y son los números totales de nodos en cada dirección. En la Figura 2 se muestra la discretización resultante, junto con el stencil que conecta cada nodo interior con sus cuatro vecinos.

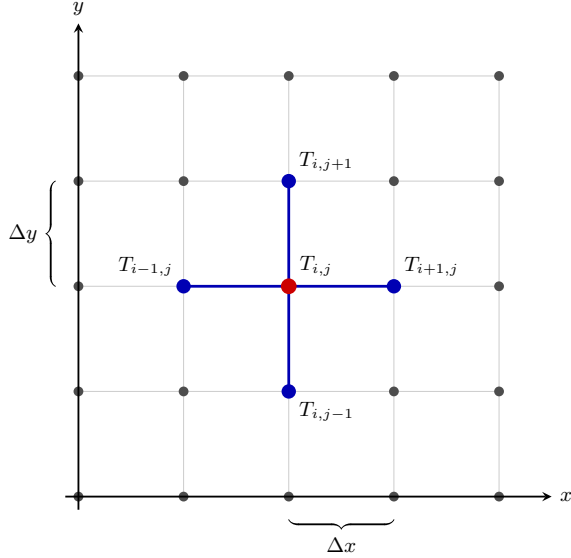


Figura 2: Discretización del dominio bidimensional y stencil de cinco puntos asociado a la ecuación (14).

Para el caso bidimensional en el rectángulo $[0, L_x] \times [0, L_y]$ con condiciones de Dirichlet en los cuatro lados, los valores $T_{i,j}$ se reordenan en un vector unidimensional mediante la numeración lexicográfica $k = i + j N_x$, reduciendo el sistema a la forma $A \mathbf{T} = \mathbf{b}$ con $\mathbf{T} \in \mathbb{R}^{N_x N_y}$. Para una malla de $N_x = N_y = 4$ nodos por lado, la matriz A tiene estructura *bloque-tridiagonal*

$$A = \begin{pmatrix} I & 0 & 0 & 0 \\ I & B & I & 0 \\ 0 & I & B & I \\ 0 & 0 & 0 & I \end{pmatrix}, \quad B = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 1 & -4 & 1 & 0 \\ 0 & 1 & -4 & 1 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \quad (15)$$

donde los bloques I de identidad imponen las condiciones de frontera y B codifica el stencil de cinco puntos en cada fila de la malla. El vector $\mathbf{b}$ contiene los valores prescritos en los nodos frontera y ceros en los interiores.

Para condiciones de Neumann, las filas frontera de A se reemplazan por las aproximaciones en diferencias adelantadas o atrasadas de la deriva-

da normal, y $\mathbf{b}$ recibe los flujos correspondientes multiplicados por h .

2.4. Resolución del sistema lineal

2.4.1. Caso 1D

La matriz tridiagonal del sistema (9) admite una inversión directa en $\mathcal{O}(N)$ operaciones mediante el *algoritmo de Thomas*, una variante de la eliminación gaussiana especializada para matrices tridiagonales. El método consta de dos barridos: uno hacia adelante para eliminar la subdiagonal y normalizar las ecuaciones, y otro hacia atrás para recuperar el vector solución por sustitución regresiva. Su complejidad óptima y su bajo consumo de memoria lo convierten en la elección estándar para este tipo de sistemas.

2.4.2. Caso 2D: método ADI

En dos dimensiones, la matriz del sistema discretizado tiene tamaño $N_x N_y \times N_x N_y$, por lo que su inversión directa no es viable. Se emplea entonces el *método ADI* (*Alternating Direction Implicit*), que divide cada iteración en dos semi-barridos: uno implícito en x y explícito en y , y otro con los roles invertidos. Cada semi-barrido produce un conjunto de sistemas tridiagonales independientes, N_y sistemas de tamaño N_x en el primer paso, y N_x de tamaño N_y en el segundo, resueltos eficientemente con el algoritmo de Thomas. El coste por iteración escala como $\mathcal{O}(N_x N_y)$.

El método es naturalmente paralelizable, ya que los sistemas de un mismo semi-barrido pueden resolverse de forma simultánea, requiriendo únicamente que todos ellos finalicen antes de intercambiar la dirección implícita. La iteración prosigue hasta alcanzar la tolerancia ε preestablecida, controlándose la convergencia mediante el parámetro iterativo $\Delta\tau$, cuyo valor, sin significado físico en el régimen estacionario, puede optimizarse con secuencias cíclicas. En la Figura /citefig:sim se observa el resultado obtenido usando este método de resolución.

3. Implementación

La implementación se realizó en **Fortran**, utilizando las API de OpenMP y OpenACC, que facilitan la paralelización de código secuencial al distribuir la carga de cómputo entre múltiples hilos que operan simultáneamente.



Figura 3: Resultado de la resolución de la ecuación de calor en 2D usando el método ADI, después de 10000 interacciones.

En el siguiente bloque de código se muestra un ejemplo sencillo de la estructura de paralelización de una sección del código mediante OpenMP:

```
1 !omp parallel
2 ... operaciones ejecutadas por
  cada hilo
3 !omp end parallel
```

y su equivalente para OpenACC:

```
1 !acc parallel
2 ... operaciones ejecutadas por
  cada hilo
3 !acc end parallel
```

En la implementación se distribuye la carga de trabajo entre múltiples hilos que ejecutan operaciones de forma simultánea. El caso más común es la paralelización de un ciclo, que en OpenMP se logra con una sola directiva:

```
1 !omp parallel do
2   do i = 1, N
3     ... operacion sobre el
      elemento i
4   end do
5 !omp end parallel do
```

Cada hilo toma un subconjunto de iteraciones del ciclo y las ejecuta de forma independiente. Cabe señalar, sin embargo, que es necesario evitar las condiciones de carrera: situaciones en las que dos o más hilos acceden y modifican el mismo bloque de memoria de forma simultánea, produciendo resultados no deseados. En el contexto del método ADI, esto no representa un problema ya que los sistemas tridiagonales de cada semi-barrido son completamente independientes entre sí.

4. Resultados

4.1. Rendimiento del código paralelo

Para evaluar la ganancia obtenida al paralelizar el código, se compara el tiempo de ejecución del programa en paralelo frente al serial (un solo hilo) mediante dos cantidades estándar en el computo paralelo.

Speedup: mide qué tan rápido se ejecuta el programa en paralelo en comparación con la versión serial, y se define como la razón entre los tiempos de ejecución,

$$S(p) = \frac{t_s}{t_p}, \quad (16)$$

donde t_s es el tiempo de ejecución serial y t_p es el tiempo de ejecución con p hilos. Un speedup ideal es $S(p) = p$, es decir, duplicar el número de hilos reduce a la mitad el tiempo de ejecución. En la práctica, la fracción del código que no puede paralelizarse limita el speedup máximo alcanzable.

Eficiencia: mide qué fracción del speedup ideal se aprovecha efectivamente, y se define como

$$E(p) = \frac{S(p)}{p} = \frac{t_s}{p t_p}. \quad (17)$$

Una eficiencia $E = 1$ indica un uso perfecto de los p hilos; valores menores indican que no todos los hilos están trabajando de manera eficiente. En la Figura 4 y Figura 5 se muestran $S(p)$ y $E(p)$, respectivamente, en función del número de hilos para una malla de 420×420 nodos. Cabe recalcar que las pruebas de rendimiento fueron realizadas en un procesador intel Intel(R) Core(TM) i5-8250U CPU 1.60GHz.

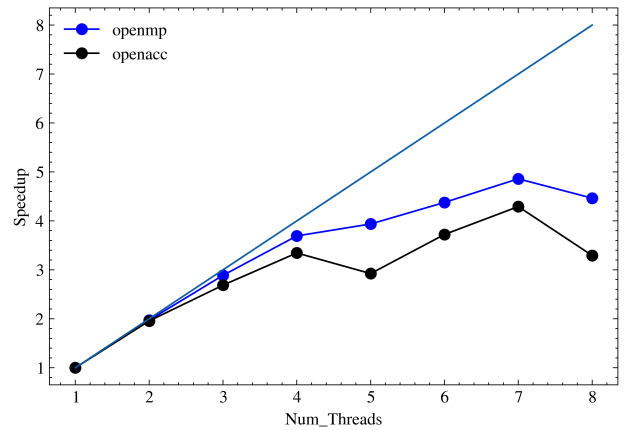


Figura 4: Speedup $S(p)$ para las distintas API de paralelización. La línea discontinua indica el speedup ideal $S = p$.

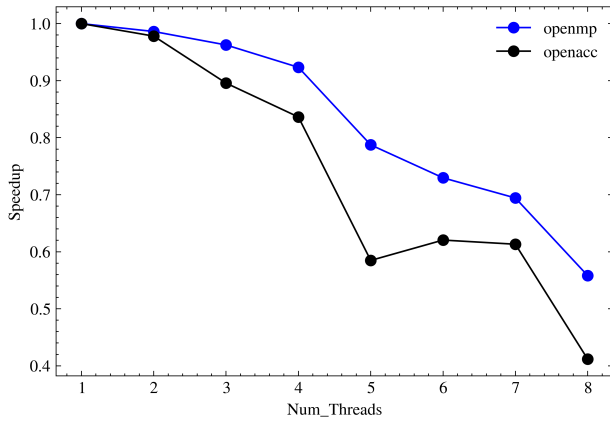


Figura 5: Eficiencia $E(p)$ para las distintas API de paralelización.

Se observa una mejora considerable al usar 4 hilos en ambas APIs de paralelización. La implementación con OpenMP se mantiene más cercana al caso ideal en términos de speedup, mientras que la implementación con OpenACC, aunque muestra cierta mejoría, se aleja más de dicho caso ideal, lo que sugiere que OpenACC no está optimizado para el tamaño de malla utilizado en la CPU.

Al utilizar más de 4 hilos, ambas APIs mantienen un rendimiento similar al observado con 4 hilos. Sin embargo, el cambio más notable se da en la eficiencia: en este aspecto, la implementación con OpenMP destaca claramente, alcanzando un valor superior a 0.9 hasta el límite de 4 hilos, punto a partir del cual se observa un cambio significativo en la eficiencia.

En cuanto a la implementación realizada con OpenACC, la eficiencia se ve afectada de manera drástica a partir del límite de 2 hilos, lo que indica que los hilos de trabajo no son bien aprovechados en esta implementación. Esta última, sin embargo, podría funcionar mejor en una GPU, ya que su forma de paralelizar con directivas y su capacidad para enviar trabajo pesado a los aceleradores le permitirían rendir más que en la CPU.

5. Conclusiones

Ambas APIs, OpenMP y OpenACC, mostraron una mejora considerable al usar 4 hilos. Sin embargo, OpenMP se mantuvo más cerca del caso ideal de speedup, mientras que OpenACC quedó más alejada, lo que sugiere que nuestra implementación no es tan buena para el uso de múltiples hilos de trabajo. OpenMP alcanzó una eficiencia superior a 0.9 hasta los 4 hilos, demostrando un excelente aprovechamiento de los núcleos. A partir de 4 hilos, la eficiencia comenzó a caer significativamente, lo cual indica que no siempre el uso de muchos hilos de trabajo implica un uso óptimo de los recursos. Por otro lado, la eficiencia de OpenACC se vio afectada por la CPU, pero se espera que en una arquitectura de GPU pueda funcionar de manera más óptima.

En conclusión, para ejecuciones en CPU con múltiples hilos, OpenMP es claramente la mejor opción, mientras que para entornos con GPU, OpenACC podría ser más conveniente, aunque se recomienda validarlo con pruebas específicas en dicho hardware.

Referencias

- [1] OpenMP Architecture Review Board. *OpenMP: The OpenMP API Specification for Parallel Programming*. <https://www.openmp.org/>. Consultado: junio 2026. 2024.
- [2] OpenACC Organization. *OpenACC: Directives for Accelerators*. <https://www.openacc.org/>. Consultado: junio 2026. 2024.
- [3] T. J. Chung. *Computational Fluid Dynamics*. 1.^a ed. Cambridge: Cambridge University Press, 2002. ISBN: 9780521594165.
- [4] S. V. Patankar. *Numerical Heat Transfer and Fluid Flow*. 1.^a ed. CRC Press, 1980. ISBN: 9780891165224.